Boolean Expression Simplifier with Quine-McCluskey Algorithm

Lamri Mohamed Yassine

April 8, 2025

1 Introduction

This report analyzes a Python implementation of a Boolean expression simplifier that uses the Quine-McCluskey algorithm. The system consists of two main components: a graphical user interface (GUI) built with Textual and a core logic module for Boolean algebra operations.

2 System Architecture

2.1 Graphical User Interface

The GUI (`gui.py`) provides an interactive environment for users to:

- Input Boolean expressions in sum-of-products form (e.g., `x*y + ~x*z`)
- Simplify expressions using the Quine-McCluskey algorithm
- Generate truth tables for expressions
- Clear inputs and outputs

Key features of the GUI implementation:

- Built using Textual, a TUI (Text User Interface) framework
- Clean, dark-themed interface with proper widget organization
- Responsive layout with clear visual feedback
- Separation of presentation logic from core functionality

2.2 Core Logic

The `core.py` module implements:

- Boolean expression parsing and evaluation
- Truth table generation
- Expression simplification using Quine-McCluskey

3 Boolean Algebra Modeling

The system models Boolean expressions using two main classes:

3.1 Term Class

```
class Term:
    def __init__(self, symbol: str, is_negated: bool = False):
        self.symbol = symbol.strip()
        self.is_negated = is_negated

    def evaluate(self, inputs: Dict[str, bool]) -> bool:
        value = inputs.get(self.symbol, False)
        return not value if self.is_negated else value
```

This class represents individual terms in Boolean expressions, handling both regular and negated terms.

3.2 BooleanExpression Class

```
class BooleanExpression:
    def __init__(self, expression: str):
        self.raw_expression = expression
        self.terms = self._parse_expression(expression)
```

This class manages the entire Boolean expression, including parsing, evaluation, and simplification.

4 Quine-McCluskey Implementation

The algorithm is implemented in three main phases:

4.1 1. Finding Prime Implicants

Algorithm 1 Find Prime Implicants

```
Group minterms by number of 1s in their binary representation
while new combinations can be made do
    Compare terms in adjacent groups
    Combine terms that differ by exactly one bit
    Mark used terms
    Add unmarked terms to prime implicants
end while
```

4.2 2. Identifying Essential Primes

Algorithm 2 Find Essential Primes

```
for each minterm do
    Count how many primes cover it
    if minterm is covered by only one prime then
        Add to essential primes
    end if
end for
Select additional primes to cover remaining minterms
```

4.3 3. Formatting Simplified Expression

The final step converts the prime implicants back to Boolean expression form.

5 Complexity Analysis

The Quine-McCluskey algorithm has the following complexity characteristics:

- **Time Complexity**: $O(3^n/n)$ in worst case
- **Space Complexity**: $O(2^n)$ for storing minterms
- **Practical Considerations**:
  - Efficient for expressions with $\leq$ 4-5 variables
  - Becomes impractical for expressions with many variables

6 Interactive prompt

There is the option to prompt boolean expression in order to simplify them in an easy way with examples provided.

7 Conclusion

The implementation provides a complete solution for Boolean expression simplification with:

- Clean separation between UI and core logic
- Accurate implementation of Quine-McCluskey algorithm
- User-friendly interface for expression input and results visualization

Future improvements could include:

- Handling of don't-care conditions
- Alternative simplification methods for larger expressions
- Extended Boolean algebra operations